



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de **HONORIS UNITED UNIVERSITIES**

ExpertLink Intelligent

Plateforme de Cartographie des Connaissances

Découverte d'Experts & Parcours d'Apprentissage

Rapport de Projet Big Data

Réalisé par

Omar SOUBAIBI

Mohamed Reda CHOUIKH

Wadi MOHAMED

Encadré par

Mme. Fatima Ezzahra MDARBI

Année Universitaire 2025/2026

Dédicace

*À nos chers parents,
pour leur soutien indéfectible et leurs sacrifices*

*À nos professeurs,
pour leur guidance et leur patience*

*À nos familles et amis,
pour leurs encouragements constants*

Nous vous dédions ce travail.

**Omar SOUBAIBI
Mohamed Reda CHOUIKH
Wadi MOHAMED**

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

En premier lieu, nous adressons nos sincères remerciements à notre encadrante, **Madame Fatima Ezzahra MDARBI**, pour son accompagnement précieux, ses conseils avisés et sa disponibilité tout au long de ce projet. Sa rigueur scientifique et son expertise technique nous ont été d'une aide inestimable.

Nous remercions également l'ensemble du corps professoral de l'**École Marocaine des Sciences de l'Ingénieur (EMSI)** pour la qualité de l'enseignement dispensé et pour nous avoir transmis les connaissances nécessaires à la réalisation de ce projet.

Nos remerciements vont aussi à nos familles respectives pour leur soutien moral et financier, leur patience et leurs encouragements durant toute notre formation.

Enfin, nous tenons à remercier tous nos camarades de promotion pour les moments de partage, d'entraide et de collaboration qui ont enrichi notre parcours académique.

Omar SOUBAIBI, Mohamed Reda CHOUIKH, Wadi MOHAMED

Résumé

Ce projet de fin d'année présente la conception et le développement d'ExpertLink Intelligent, une plateforme innovante de cartographie des connaissances destinée aux organisations modernes. Face aux défis croissants de gestion des talents et des compétences dans les entreprises, notre solution propose une approche novatrice basée sur les technologies de bases de données orientées graphe.

ExpertLink permet d'identifier rapidement les experts au sein d'une organisation, de visualiser les relations complexes entre compétences, projets et collaborateurs, et de générer des parcours d'apprentissage personnalisés. L'architecture technique repose sur trois piliers : un backend performant développé avec FastAPI (Python), une interface utilisateur moderne construite avec React et TypeScript, et une base de données Neo4j pour la modélisation des graphes de connaissances.

Les fonctionnalités principales incluent la recherche avancée d'experts, la recherche plein texte de documents, une visualisation 3D interactive du réseau de connaissances, des recommandations de parcours d'apprentissage basées sur l'analyse des prérequis, et un tableau de bord analytique complet.

Ce rapport détaille l'ensemble du processus de développement, depuis l'analyse des besoins jusqu'aux tests de validation, en passant par les choix architecturaux et les décisions techniques.

Abstract :

This end-of-year project presents the design and development of ExpertLink Intelligent, an innovative knowledge mapping platform designed for modern organizations. Facing growing challenges in talent and skills management within companies, our solution offers an innovative approach based on graph database technologies.

ExpertLink enables rapid identification of experts within an organization, visualization of complex relationships between skills, projects, and collaborators, and generation of personalized learning paths. The technical architecture is built on three pillars : a high-performance backend developed with FastAPI (Python), a modern user interface built with React and TypeScript, and a Neo4j database for knowledge graph modeling.

Key features include advanced expert search, full-text document search, interactive 3D visualization of the knowledge network, learning path recommendations based on prerequisite analysis, and a comprehensive analytics dashboard.

Table des matières

Remerciements	2
Résumé	3
1 Introduction Générale	10
1.1 Contexte Général	10
1.2 Problématique	11
1.3 Objectifs du Projet	12
1.4 Méthodologie de Travail	13
1.5 Plan du Rapport	14
1.6 Technologies Utilisées	15
2 État de l’Art	16
2.1 Introduction	16
2.2 Les Bases de Données Orientées Graphe	17
2.3 Frameworks Web Modernes	18
2.4 Solutions de Gestion des Connaissances Existantes	19
2.5 Positionnement d’ExpertLink	20
3 Architecture du Système	21
3.1 Vue d’Ensemble	21
3.2 Structure du Projet	22
3.3 Communication entre Couches	23
3.4 Patterns de Conception Utilisés	24
3.5 Flux de Données Complet	25
3.6 Gestion des Erreurs	26
4 Base de Données Neo4j	27
4.1 Pourquoi Neo4j ?	27
4.2 Schéma de Données	28
4.3 Exemples de Requêtes Cypher	28
4.4 Visualisation du Schéma de Graphe	31
4.5 Statistiques de la Base de Données	32

5	Backend FastAPI	33
5.1	Présentation de FastAPI	33
5.2	Point d'Entrée de l'Application	34
5.3	Endpoints API	35
5.4	Authentification JWT	36
6	Frontend React	37
6.1	Architecture du Frontend	37
6.2	Composant Principal	38
6.3	Thème Personnalisé	39
6.4	Composants Réutilisables	40
6.5	Gestion de l'État	41
6.6	Visualisation 3D	42
6.7	Responsive Design	43
7	Fonctionnalités de l'Application	44
7.1	Page de Connexion	44
7.2	Recherche d'Experts	45
7.3	Recherche de Documents	46
7.4	Visualisation du Graphe de Connaissances	47
7.5	Parcours d'Apprentissage	48
7.6	Tableau de Bord Analytique	49
8	Guide d'Installation	50
8.1	Prérequis	50
8.2	Installation de Neo4j	51
8.3	Installation du Backend	52
8.4	Installation du Frontend	53
8.5	Variables d'Environnement	54
8.6	Accès aux Services	55
9	Données du Système	56
9.1	Format des Données	56
9.2	Exemple de Données Expert	57
9.3	Génération de Données	58
10	Sécurité et Performance	59
10.1	Introduction à la Sécurité	59
10.2	Authentification et Autorisation	60
10.3	Protection des Données	61
10.4	Sécurité du Frontend	62
10.5	Performance et Optimisation	63

10.6 Bonnes Pratiques de Sécurité	64
11 Tests et Validation	65
11.1 Tests du Backend	65
11.2 Vérification de Santé	66
11.3 Documentation Interactive	67
12 Conclusion et Perspectives	68
12.1 Bilan du Projet	68
12.2 Points Forts de la Solution	70
12.3 Défis Rencontrés et Solutions	71
12.4 Compétences Acquisées	72
12.5 Perspectives d'Évolution	73
12.6 Conclusion Générale	74
A Annexe : Guide de Démarrage Rapide	75
A.1 Étapes de Démarrage	76
B Bibliographie et Webographie	77
B.1 Bibliographie	77
B.2 Webographie	78
C Annexe : Glossaire	79

Table des figures

2.1	Positionnement d'ExpertLink face à la concurrence	20
3.1	Architecture en trois couches d'ExpertLink	21
3.2	Flux de données dans l'application ExpertLink	25
4.1	Schéma du graphe de connaissances ExpertLink	31
7.1	Page de connexion ExpertLink	44
7.2	Interface de recherche d'experts	45
7.3	Interface de recherche de documents	46
7.4	Visualisation 3D du graphe de connaissances	47
7.5	Interface des parcours d'apprentissage	48
7.6	Tableau de bord analytique	49

Liste des tableaux

1.1	Stack Technologique du Projet	15
2.1	Comparaison : Base Relationnelle vs Base Graphe	17
2.2	Alternatives aux Bases de Données Graphe	17
2.3	Comparaison des Frameworks Python Backend	18
3.1	Stratégie de Gestion des Erreurs	26
4.1	Types de Nœuds dans Neo4j	28
4.2	Relations entre les Nœuds	28
4.3	Statistiques de la Base de Données ExpertLink	32
5.1	Endpoints d'Authentification	35
5.2	Endpoints de Gestion des Experts	35
5.3	Endpoints de Gestion des Documents	35
5.4	Endpoints du Graphe de Connaissances	35
5.5	Endpoints des Parcours d'Apprentissage	36
8.1	URLs des Services	55
10.1	Matrice des Permissions par Rôle	60
12.1	Bilan des Objectifs du Projet	68

Chapitre 1

Introduction Générale

1.1 Contexte Général

À l'ère du numérique et de la transformation digitale, les organisations font face à des défis sans précédent en matière de gestion des connaissances et des compétences. La croissance exponentielle des données, la complexification des métiers et la mobilité accrue des talents rendent obsolètes les approches traditionnelles de gestion des ressources humaines.

Dans ce contexte, l'identification rapide des expertises, la visualisation des réseaux de compétences et l'optimisation des parcours de formation deviennent des enjeux stratégiques majeurs pour toute organisation souhaitant rester compétitive.

1.2 Problématique

Les entreprises modernes rencontrent plusieurs difficultés majeures dans la gestion de leur capital humain. Le **silotage des connaissances** constitue un premier obstacle majeur : les compétences sont souvent fragmentées entre départements, rendant difficile l'identification des experts transverses capables d'intervenir sur des projets pluridisciplinaires.

La **perte de savoir-faire** représente également un risque considérable. Le départ d'employés clés entraîne une perte de connaissances tacites difficilement récupérables, ce qui peut impacter significativement la productivité et la qualité des livrables.

L'**inefficacité du matching** entre les besoins des projets et les compétences disponibles reste un processus long et peu fiable dans la plupart des organisations. Les responsables de projet peinent à identifier rapidement les bonnes personnes pour leurs équipes.

Enfin, les **parcours de formation** proposés sont souvent génériques et ne tiennent pas compte des prérequis individuels ni des objectifs de carrière spécifiques de chaque collaborateur.

1.3 Objectifs du Projet

Face à ces problématiques, le projet ExpertLink Intelligent vise à développer une plateforme complète de gestion des connaissances. Le premier objectif consiste à **cartographier les connaissances** en modélisant l'ensemble des relations entre experts, compétences, projets et documents au sein d'un graphe exploitable.

La plateforme doit également **faciliter la découverte d'experts** en proposant des outils de recherche avancés permettant d'identifier rapidement les profils correspondant à des critères spécifiques. Une interface de **visualisation 3D** intuitive permettra d'explorer le graphe de connaissances de manière interactive.

Le système devra aussi **personnaliser l'apprentissage** en générant des parcours de formation adaptés aux besoins et au niveau de chaque collaborateur, tout en fournissant des **tableaux de bord analytiques** pour suivre l'évolution des compétences.

1.4 Méthodologie de Travail

Pour mener à bien ce projet, nous avons adopté une méthodologie agile structurée en plusieurs phases. La **phase d'analyse** a permis d'étudier les besoins, de réaliser une veille technologique et de définir les spécifications fonctionnelles du système.

La **phase de conception** a consisté en l'élaboration de l'architecture technique, la modélisation des données et le design des interfaces utilisateur. L'implémentation s'est déroulée de manière itérative lors de la **phase de développement**, avec des tests continus pour garantir la qualité du code.

La **phase de test** a permis la validation fonctionnelle, les tests de performance et les corrections nécessaires. Enfin, la **phase de documentation** a abouti à la rédaction de ce rapport et de la documentation technique.

1.5 Plan du Rapport

Ce rapport est organisé en dix chapitres. Le premier chapitre présente le contexte, la problématique et les objectifs du projet. Le deuxième chapitre propose un état de l’art des solutions existantes et des technologies utilisées.

Le troisième chapitre détaille l’architecture technique du système, suivi du quatrième chapitre consacré à la base de données Neo4j. Les cinquième et sixième chapitres présentent respectivement le backend FastAPI et le frontend React.

Le septième chapitre décrit les fonctionnalités de l’application avec des captures d’écran. Le huitième chapitre aborde les aspects de sécurité et de performance. Le neuvième chapitre fournit le guide d’installation et les procédures de test. Enfin, le dixième chapitre conclut avec un bilan et les perspectives d’évolution.

1.6 Technologies Utilisées

Le projet s'appuie sur une stack technologique moderne et performante :

TABLE 1.1 – Stack Technologique du Projet

Composant	Technologies
Backend	Python 3.11+, FastAPI, Pydantic, python-jose (JWT), Uvicorn
Frontend	React 18, TypeScript 5.0+, Material-UI (MUI), react-force-graph-3d, Recharts, Vite
Base de données	Neo4j 5.x avec langage Cypher
Outils de développement	Git, VS Code, Docker, npm

Chapitre 2

État de l'Art

2.1 Introduction

Avant de développer notre solution, il était essentiel d'étudier les approches existantes en matière de gestion des connaissances et de découverte d'experts. Ce chapitre présente une analyse des technologies et solutions disponibles sur le marché.

2.2 Les Bases de Données Orientées Graphe

2.2.1 Définition et Principes

Une base de données orientée graphe est un système de gestion de base de données qui utilise la théorie des graphes pour stocker, cartographier et interroger les relations entre les données. Contrairement aux bases de données relationnelles classiques, elles excellent dans la modélisation des relations complexes.

TABLE 2.1 – Comparaison : Base Relationnelle vs Base Graphe

Critère	Base Relationnelle	Base Graphe
Modèle de données	Tables, lignes, colonnes	Nœuds, relations, propriétés
Jointures	Coûteuses en performance	Natives et efficaces
Requêtes de parcours	Complexes et lentes	Optimisées et rapides
Flexibilité du schéma	Rigide	Flexible
Cas d'usage idéal	Données tabulaires	Données connectées

2.2.2 Neo4j : Leader du Marché

Neo4j est la base de données orientée graphe la plus populaire au monde. Ses caractéristiques principales incluent :

- **Modèle de propriétés** : Nœuds et relations peuvent avoir des propriétés
- **Langage Cypher** : Langage de requête déclaratif et expressif
- **ACID Compliance** : Garantie de transactions atomiques, cohérentes, isolées et durables
- **Scalabilité** : Support du clustering pour les déploiements à grande échelle
- **Écosystème riche** : Drivers officiels pour de nombreux langages

2.2.3 Alternatives à Neo4j

D'autres solutions de bases de données graphe existent sur le marché :

TABLE 2.2 – Alternatives aux Bases de Données Graphe

Solution	Avantages	Inconvénients
Amazon Neptune	Intégration AWS, serverless	Vendor lock-in
ArangoDB	Multi-modèle, open source	Moins mature que Neo4j
JanusGraph	Distribué, open source	Configuration complexe
OrientDB	Multi-modèle, SQL-like	Communauté plus restreinte

2.3 Frameworks Web Modernes

2.3.1 Backend : FastAPI vs Alternatives

Pour le développement du backend, plusieurs frameworks Python ont été évalués :

TABLE 2.3 – Comparaison des Frameworks Python Backend

Framework	Performance	Documentation	Adoption
FastAPI	Excellente	Automatique	Croissante
Django	Bonne	Manuelle	Très large
Flask	Bonne	Minimale	Large
Starlette	Excellente	Minimale	Moyenne

FastAPI a été choisi pour ses performances exceptionnelles et sa documentation automatique (Swagger/OpenAPI).

2.3.2 Frontend : React vs Alternatives

Pour l'interface utilisateur, React a été sélectionné parmi les alternatives suivantes :

- **React** : Bibliothèque flexible avec un large écosystème (choisi)
- **Vue.js** : Framework progressif, plus simple mais moins d'écosystème
- **Angular** : Framework complet mais courbe d'apprentissage élevée
- **Svelte** : Performances excellentes mais écosystème limité

2.4 Solutions de Gestion des Connaissances Existantes

2.4.1 Solutions Commerciales

Plusieurs solutions commerciales existent sur le marché de la gestion des connaissances :

- **Confluence (Atlassian)** : Wiki d'entreprise avec collaboration
- **SharePoint (Microsoft)** : Plateforme de gestion documentaire
- **Notion** : Outil collaboratif polyvalent
- **Guru** : Base de connaissances avec IA

2.4.2 Limites des Solutions Existantes

Les solutions existantes présentent plusieurs limitations :

1. **Manque de visualisation graphique** : Peu de solutions offrent une représentation visuelle des relations
2. **Découverte d'experts limitée** : La recherche est souvent textuelle et peu contextuelle
3. **Parcours d'apprentissage absents** : Pas de génération automatique de parcours personnalisés
4. **Silos d'information** : Les connaissances restent fragmentées entre outils

2.5 Positionnement d'ExpertLink

ExpertLink se distingue par sa combinaison unique de fonctionnalités :

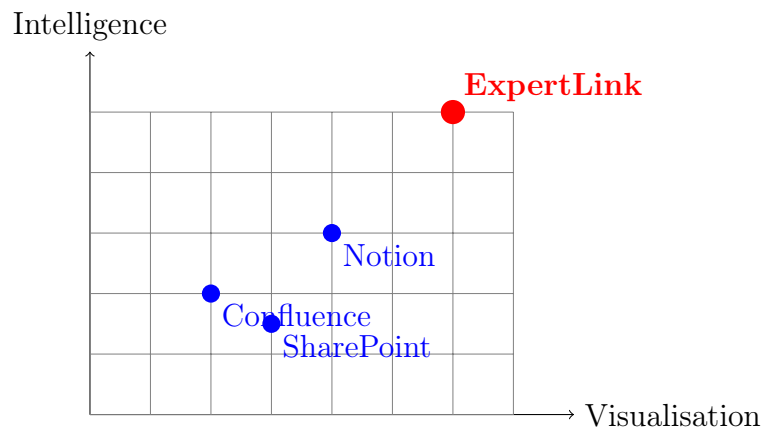


FIGURE 2.1 – Positionnement d'ExpertLink face à la concurrence

Chapitre 3

Architecture du Système

3.1 Vue d'Ensemble

L'architecture d'ExpertLink suit un modèle **client-serveur** avec trois couches principales :

1. **Couche Présentation (Frontend)** : Application React avec Material-UI
2. **Couche Métier (Backend)** : API RESTful FastAPI
3. **Couche Données** : Base de données graphe Neo4j

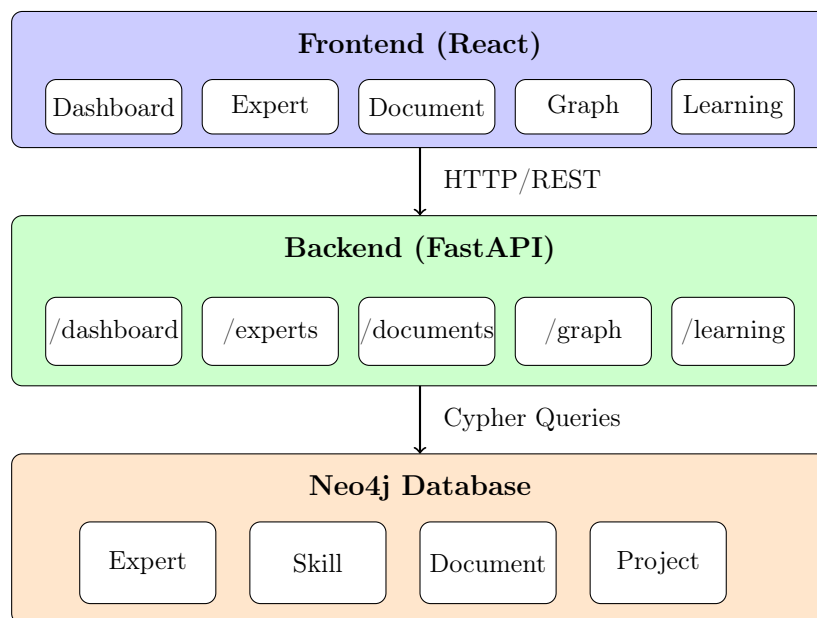


FIGURE 3.1 – Architecture en trois couches d'ExpertLink

3.2 Structure du Projet

Le projet est organisé de manière modulaire :

```
1 ExpertLink/  
2 +-- backend/  
3 |   +-- app/  
4 |   |   +-- main.py           # Point d'entree FastAPI  
5 |   |   +-- database.py       # Connexion Neo4j  
6 |   |   +-- config.py         # Configuration  
7 |   |   +-- auth_utils.py     # Authentification JWT  
8 |   |   +-- models/           # Modeles Pydantic  
9 |   |   +-- routers/          # Endpoints API  
10 |   +-- requirements.txt  
11 +-- frontend/  
12 |   +-- src/  
13 |   |   +-- App.tsx           # Composant principal  
14 |   |   +-- pages/            # Pages de l'application  
15 |   |   +-- services/         # Client API  
16 |   |   +-- theme/            # Theme Material-UI  
17 |   +-- package.json  
18 +-- data/                     # Donnees JSONL  
19 +-- rapport/                  # Documentation
```

Listing 3.1 – Structure du projet ExpertLink

3.3 Communication entre Couches

3.3.1 Frontend vers Backend

Le frontend communique avec le backend via des requêtes HTTP REST utilisant Axios :

```
1 import axios from 'axios';
2
3 const api = axios.create({
4   baseURL: 'http://localhost:8000',
5   headers: {
6     'Content-Type': 'application/json',
7   },
8 });
9
10 // Interception pour ajouter le token JWT
11 api.interceptors.request.use((config) => {
12   const token = localStorage.getItem('token');
13   if (token) {
14     config.headers.Authorization = 'Bearer ${token}';
15   }
16   return config;
17 });
```

Listing 3.2 – Service API Frontend (api.ts)

3.3.2 Backend vers Base de Données

Le backend utilise le driver officiel Neo4j pour Python :

```
1 from neo4j import GraphDatabase
2
3 class Neo4jDriver:
4     _driver = None
5
6     @classmethod
7     def get_driver(cls):
8         if cls._driver is None:
9             cls._driver = GraphDatabase.driver(
10                 settings.neo4j_uri,
11                 auth=(settings.neo4j_user, settings.neo4j_password)
12             )
13         return cls._driver
```

Listing 3.3 – Connexion Neo4j (database.py)

3.4 Patterns de Conception Utilisés

L'architecture d'ExpertLink implémente plusieurs design patterns reconnus pour garantir la qualité et la maintenabilité du code.

3.4.1 Pattern Singleton

Le driver Neo4j utilise le pattern Singleton pour garantir une instance unique de connexion à la base de données. Ce pattern permet d'économiser les ressources en évitant les connexions multiples. La variable de classe est initialisée une seule fois lors du premier appel, et les accès concurrents sont gérés de manière thread-safe par le driver Neo4j lui-même.

3.4.2 Pattern Repository

Les routers FastAPI suivent le pattern Repository pour séparer clairement la logique métier de l'accès aux données. Chaque endpoint API délègue les opérations de base de données à des fonctions spécialisées qui encapsulent les requêtes Cypher. Cette séparation facilite les tests unitaires et permet de modifier l'implémentation de la persistance sans impacter la logique applicative.

3.4.3 Pattern Factory

La création des objets de réponse API utilise le pattern Factory via Pydantic. Les modèles Pydantic définissent la structure des données et assurent la validation automatique. La conversion depuis les résultats Neo4j vers les objets de réponse est réalisée de manière déclarative, ce qui simplifie le code et réduit les risques d'erreurs.

3.5 Flux de Données Complet

Le flux de données dans l'application suit un chemin bien défini :

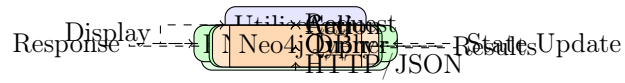


FIGURE 3.2 – Flux de données dans l'application ExpertLink

3.6 Gestion des Erreurs

L'application implémente une gestion des erreurs à plusieurs niveaux :

TABLE 3.1 – Stratégie de Gestion des Erreurs

Couche	Type d'Erreur	Traitement
Frontend	Erreurs réseau	Message utilisateur + retry
Frontend	Erreurs de validation	Affichage champs en erreur
Backend	Validation Pydantic	HTTP 422 + détails
Backend	Non authentifié	HTTP 401 + redirection
Backend	Non autorisé	HTTP 403 + message
Backend	Ressource non trouvée	HTTP 404
Database	Connexion échouée	Fallback + mode dégradé

Chapitre 4

Base de Données Neo4j

4.1 Pourquoi Neo4j ?

Neo4j est une base de données orientée graphe qui excelle dans la modélisation et l'interrogation de données hautement connectées. Pour ExpertLink, ce choix s'impose naturellement car :

- Les relations entre experts, compétences et projets sont intrinsèquement un **graphe**
- Les requêtes de **parcours** (shortest path) sont natives
- La visualisation des **réseaux de connaissances** est facilitée
- Les performances restent excellentes pour les requêtes traversant de nombreuses relations

4.2 Schéma de Données

4.2.1 Nœuds (Nodes)

TABLE 4.1 – Types de Nœuds dans Neo4j

Nœud	Propriétés
Expert	id, name, email, title, department, location, years_experience, avatar
Skill	id, name, category, description
Document	id, title, content, category, author, created_at
Project	id, name, description, status, start_date, end_date

4.2.2 Relations (Relationships)

TABLE 4.2 – Relations entre les Nœuds

Relation	Description
(Expert)-[:HAS_SKILL {level}]->(Skill)	Un expert possède une compétence avec un niveau (1-5)
(Expert)-[:WORKS_ON {role}]->(Project)	Un expert travaille sur un projet avec un rôle
(Expert)-[:AUTHORED]->(Document)	Un expert est auteur d'un document
(Document)-[:RELATES_TO]->(Skill)	Un document traite d'une compétence
(Skill)-[:PREREQUISITE_OF]->(Skill)	Une compétence est prerequisite pour une autre
(Project)-[:REQUIRES]->(Skill)	Un projet nécessite une compétence

4.3 Exemples de Requêtes Cypher

4.3.1 Requête Simple

```

1 MATCH (e:Expert)
2 RETURN e.name, e.title, e.department
3 LIMIT 10

```

Listing 4.1 – Recherche de tous les experts

4.3.2 Requête de Chemin

```

1 MATCH path = shortestPath(
2   (e:Expert {name: "John Doe"}) -[*]- (s:Skill {name: "Python"})
3 )
4 RETURN path

```

Listing 4.2 – Chemin le plus court entre expert et compétence

4.3.3 Requête d’Agrégation

```

1 MATCH (e:Expert)-[:HAS_SKILL]->(s:Skill)
2 RETURN s.name AS skill, count(e) AS expert_count
3 ORDER BY expert_count DESC

```

Listing 4.3 – Nombre d’experts par compétence

4.3.4 Requête avec Filtre

```

1 MATCH (e:Expert)-[r:HAS_SKILL]->(s:Skill)
2 WHERE r.level >= 4
3 RETURN e.name, s.name, r.level

```

Listing 4.4 – Experts seniors (niveau >= 4)

4.3.5 Requête de Modification

```

1 MATCH (e:Expert {id: $expertId})-[r:HAS_SKILL]->(s:Skill {id: $skillId})
2 SET r.level = $newLevel
3 RETURN e, s

```

Listing 4.5 – Mise à jour du niveau de compétence

4.3.6 Requêtes Avancées

Recommandation d’Experts Similaires

Cette requête trouve des experts ayant des compétences similaires à un expert donné :

```

1 MATCH (e1:Expert {id: $expertId})-[:HAS_SKILL]->(s:Skill)
2     <-[:HAS_SKILL]-(e2:Expert)
3 WHERE e1 <> e2
4 WITH e2, count(s) AS shared_skills
5 ORDER BY shared_skills DESC
6 LIMIT 5
7 RETURN e2.name, e2.title, shared_skills

```

Listing 4.6 – Trouver des experts similaires

Parcours d'Apprentissage

Cette requête génère un parcours d'apprentissage basé sur les prérequis :

```
1 MATCH path = (start:Skill {name: $currentSkill})
2             -[:PREREQUISITE_OF*1..5]->
3             (goal:Skill {name: $targetSkill})
4 RETURN [node IN nodes(path) | node.name] AS learning_path,
5         length(path) AS steps
6 ORDER BY steps ASC
7 LIMIT 1
```

Listing 4.7 – Génération de parcours d'apprentissage

Analyse des Lacunes de Compétences

Identifier les compétences requises par les projets mais peu maîtrisées :

```
1 MATCH (p:Project)-[:REQUIRES]->(s:Skill)
2 OPTIONAL MATCH (e:Expert)-[r:HAS_SKILL]->(s)
3 WHERE r.level >= 4
4 WITH s, count(DISTINCT p) AS projects_needed,
5         count(DISTINCT e) AS senior_experts
6 WHERE projects_needed > senior_experts
7 RETURN s.name, projects_needed, senior_experts,
8         projects_needed - senior_experts AS gap
9 ORDER BY gap DESC
```

Listing 4.8 – Analyse des lacunes de compétences

4.4 Visualisation du Schéma de Graphe

Le schéma de données peut être visualisé comme suit :

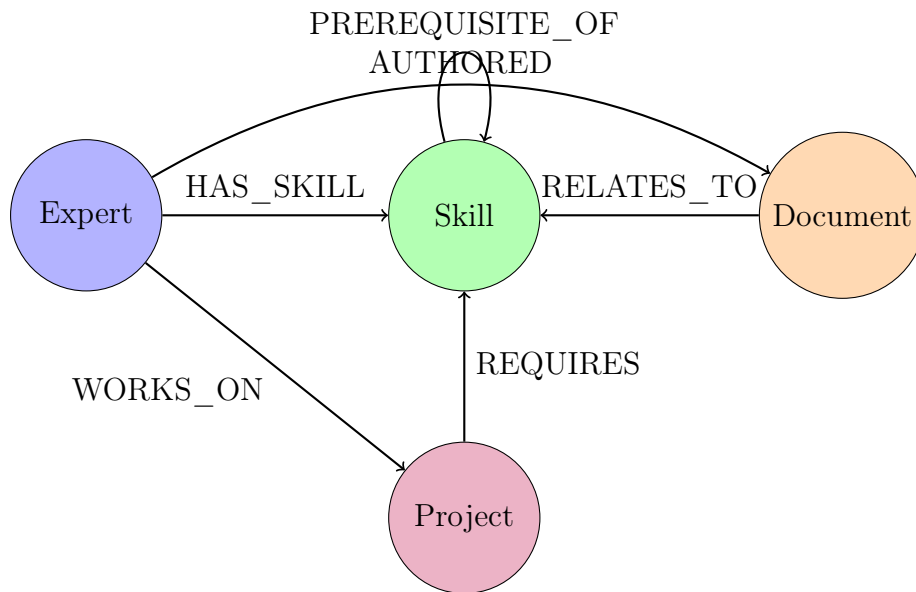


FIGURE 4.1 – Schéma du graphe de connaissances ExpertLink

4.5 Statistiques de la Base de Données

La base de données de démonstration contient les éléments suivants :

TABLE 4.3 – Statistiques de la Base de Données ExpertLink

Entité	Nombre	Description
Experts	50+	Profils d'employés avec compétences
Compétences (Skills)	40+	Taxonomie des compétences techniques et soft skills
Documents	100+	Articles, tutoriels, documentations
Projets	20+	Projets actifs et archivés
Relations HAS_SKILL	200+	Associations expert-compétence avec niveau
Relations WORKS_ON	80+	Participations aux projets
Relations PREREQUISITE_OF	30+	Dépendances entre compétences

Chapitre 5

Backend FastAPI

5.1 Présentation de FastAPI

FastAPI est un framework Python moderne et performant pour la création d'APIs. Ses avantages principaux sont :

- **Performance** : L'un des frameworks Python les plus rapides
- **Typage** : Utilisation native de Python type hints
- **Documentation automatique** : Swagger UI et ReDoc générés automatiquement
- **Validation** : Intégration de Pydantic pour la validation des données

5.2 Point d'Entrée de l'Application

```
1 from fastapi import FastAPI
2 from fastapi.middleware.cors import CORSMiddleware
3
4 app = FastAPI(
5     title="ExpertLink Intelligent",
6     description="Knowledge Cartography Platform",
7     version="1.0.0"
8 )
9
10 # Configuration CORS
11 app.add_middleware(
12     CORSMiddleware,
13     allow_origins=["http://localhost:5173"],
14     allow_credentials=True,
15     allow_methods=["*"],
16     allow_headers=["*"],
17 )
18
19 # Inclusion des routers
20 app.include_router(experts.router)
21 app.include_router(documents.router)
22 app.include_router(graph.router)
23 app.include_router(learning.router)
24 app.include_router/dashboard.router)
25 app.include_router(auth.router)
```

Listing 5.1 – Configuration principale (main.py)

5.3 Endpoints API

5.3.1 API Authentication

TABLE 5.1 – Endpoints d’Authentification

Méthode	Endpoint	Description
POST	/api/auth/login	Connexion utilisateur

5.3.2 API Experts

TABLE 5.2 – Endpoints de Gestion des Experts

Méthode	Endpoint	Description
GET	/api/experts	Liste de tous les experts
GET	/api/experts/{id}	Détail d’un expert
GET	/api/experts/search?q={query}	Recherche d’experts
GET	/api/experts/{id}/skills	Compétences d’un expert
POST	/api/experts	Créer un expert
PUT	/api/experts/{id}	Modifier un expert
DELETE	/api/experts/{id}	Supprimer un expert

5.3.3 API Documents

TABLE 5.3 – Endpoints de Gestion des Documents

Méthode	Endpoint	Description
GET	/api/documents	Liste de tous les documents
GET	/api/documents/{id}	Détail d’un document
GET	/api/documents/search?q={query}	Recherche plein texte
GET	/api/documents/category/{cat}	Filtrer par catégorie

5.3.4 API Graphe

TABLE 5.4 – Endpoints du Graphe de Connaissances

Méthode	Endpoint	Description
GET	/api/graph/nodes	Tous les nœuds du graphe
GET	/api/graph/edges	Toutes les relations
GET	/api/graph/neighbors/{id}	Voisins d’un nœud

5.3.5 API Apprentissage

TABLE 5.5 – Endpoints des Parcours d’Apprentissage

Méthode	Endpoint	Description
GET	/api/learning/paths	Parcours disponibles
GET	/api/learning/skills/{skill}/prerequisites	Prérequis d’une compétence
POST	/api/learning/recommend	Recommandations IA

5.4 Authentification JWT

L’application utilise JSON Web Tokens (JWT) pour sécuriser l’accès aux ressources protégées. Le module d’authentification implémente deux fonctions principales : la création de tokens et leur vérification.

La fonction de création génère un token signé contenant les informations de l’utilisateur et une date d’expiration fixée à 30 minutes. Ce token est encodé avec l’algorithme HS256 et une clé secrète configurée dans les variables d’environnement.

La fonction de vérification décode le token reçu dans les requêtes et valide sa signature. Si le token est invalide ou expiré, l’accès est refusé et l’utilisateur doit se reconnecter.

Chapitre 6

Frontend React

6.1 Architecture du Frontend

Le frontend est une application React moderne construite avec les technologies les plus récentes. **React 18** apporte les dernières fonctionnalités comme les hooks et le mode concurrent pour une meilleure réactivité de l'interface.

TypeScript est utilisé pour le typage statique, ce qui améliore la maintenabilité du code et réduit les erreurs à la compilation. La bibliothèque **Material-UI (MUI)** fournit une collection complète de composants UI prêts à l'emploi, conformes aux principes du Material Design de Google.

React Router gère la navigation entre les différentes pages de l'application de manière déclarative. Enfin, **Vite** est utilisé comme bundler pour un développement rapide avec rechargement à chaud instantané.

6.2 Composant Principal

Le composant principal `App.tsx` définit la structure globale de l'application. Il encapsule l'ensemble dans un `ThemeProvider` pour appliquer le thème personnalisé, un `CssBaseline` pour normaliser les styles, et un `BrowserRouter` pour la navigation.

L'interface se compose d'une barre de navigation latérale persistante et d'une zone de contenu principale. Le routage définit six pages principales : la recherche d'experts (page d'accueil), la recherche de documents, la visualisation du graphe, les parcours d'apprentissage, le tableau de bord analytique et la page de connexion.

6.3 Thème Personnalisé

L'application utilise un thème sombre moderne avec une palette de couleurs soigneusement choisie. La couleur primaire est un violet indigo (`#6366f1`) et la couleur secondaire un rose (`#f472b6`). Le fond utilise des nuances de bleu foncé (`#0f172a` pour le fond principal et `#1e293b` pour les surfaces) créant un effet de profondeur élégant.

6.4 Composants Réutilisables

L'application est structurée autour de composants React réutilisables. Le composant **Navigation** gère la barre latérale avec le menu, les liens et le logout. Les composants **ExpertCard** et **DocumentCard** affichent respectivement les informations des experts et des documents sous forme de cartes.

Le composant **SkillChip** représente les compétences sous forme de badges colorés avec leur niveau. **SearchBar** offre une barre de recherche avec filtres et suggestions. **GraphContainer** encapsule la visualisation 3D interactive. Enfin, **StatsCard** affiche les statistiques avec tendances et icônes.

6.5 Gestion de l'État

L'application utilise une combinaison de solutions pour gérer l'état de manière efficace.

6.5.1 État Local avec Hooks

Les composants utilisent les hooks `useState` et `useEffect` pour gérer l'état local. Par exemple, le composant de recherche d'experts maintient des états pour la liste des résultats, l'indicateur de chargement, la requête de recherche et les filtres actifs. L'effet `useEffect` déclenche automatiquement une nouvelle recherche lorsque la requête ou les filtres changent.

6.5.2 État Global avec Context API

Pour les données partagées comme l'authentification, l'application utilise la Context API de React. Un `AuthProvider` encapsule l'application et expose les informations de l'utilisateur connecté, les fonctions de connexion et déconnexion, ainsi qu'un indicateur d'authentification. Ce contexte permet à tous les composants d'accéder à l'état d'authentification sans prop drilling.

6.6 Visualisation 3D

La visualisation du graphe de connaissances utilise la bibliothèque **react-force-graph-3d** qui permet un rendu 3D interactif performant. Le composant charge les nœuds et les liens depuis l'API, puis configure le graphe avec des couleurs personnalisées selon le type de nœud, une couleur de fond assortie au thème et des gestionnaires d'événements pour l'interactivité.

6.7 Responsive Design

L'interface s'adapte à différentes tailles d'écran grâce au système de breakpoints de Material-UI. Les grilles utilisent des colonnes adaptatives (12 sur mobile, 6 sur tablette, 4 sur desktop, 3 sur grand écran). La barre latérale est masquée sur les petits écrans et les espacements s'ajustent automatiquement selon la taille de l'écran.

Chapitre 7

Fonctionnalités de l'Application

7.1 Page de Connexion

L'application dispose d'une interface de connexion sécurisée avec authentification JWT.

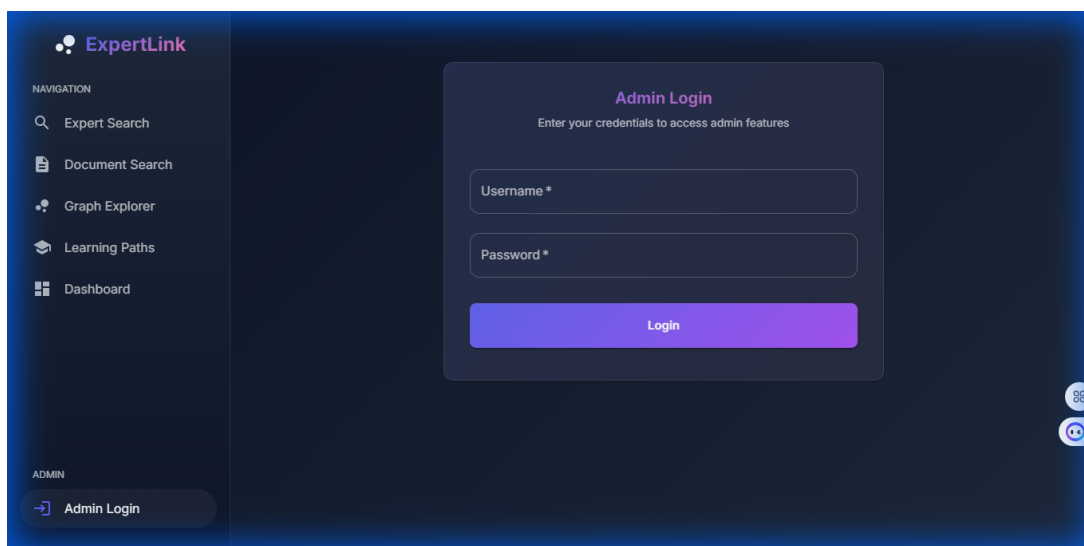


FIGURE 7.1 – Page de connexion ExpertLink

Identifiants de test :

- **Administrateur** : admin / password
- **Utilisateur** : user / password

7.2 Recherche d'Experts

La fonctionnalité de recherche d'experts permet de trouver des collaborateurs selon :

- Compétences et niveaux d'expertise (échelle 1-5)
- Localisation géographique
- Années d'expérience
- Département et rôle

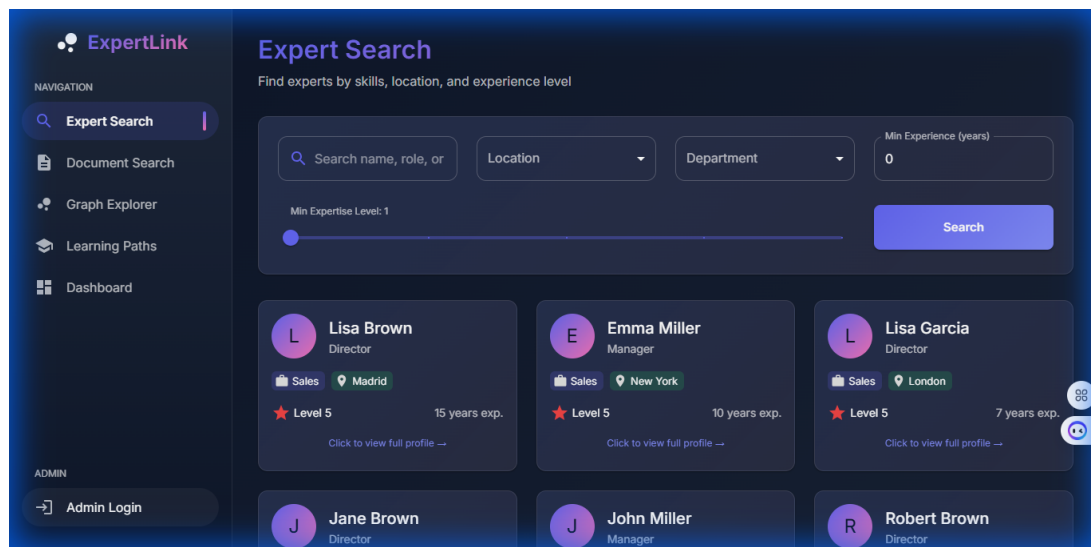


FIGURE 7.2 – Interface de recherche d'experts

7.3 Recherche de Documents

Une recherche plein texte intelligente dans les documents organisationnels avec :

- Scoring de pertinence basé sur le NLP
- Filtrage par catégorie
- Mise en surbrillance des mots-clés
- Association document-compétence

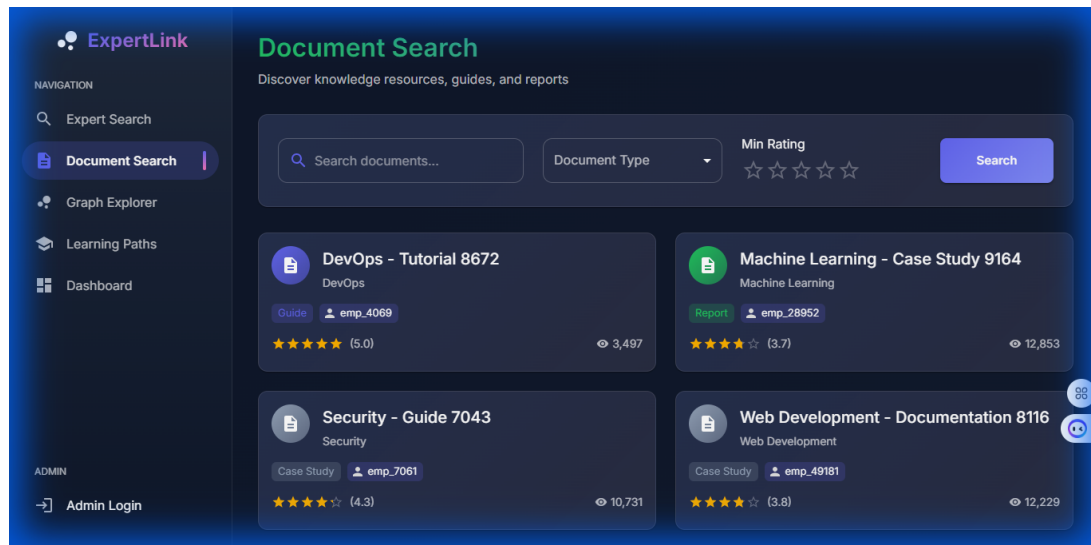


FIGURE 7.3 – Interface de recherche de documents

7.4 Visualisation du Graphe de Connaissances

L'une des fonctionnalités phares d'ExpertLink est la visualisation 3D interactive du graphe de connaissances :

- Rendu 3D en temps réel avec `react-force-graph-3d`
- Exploration des nœuds (Experts, Compétences, Documents, Projets)
- Découverte des relations
- Contrôles de zoom, panoramique et rotation
- Navigation par clic

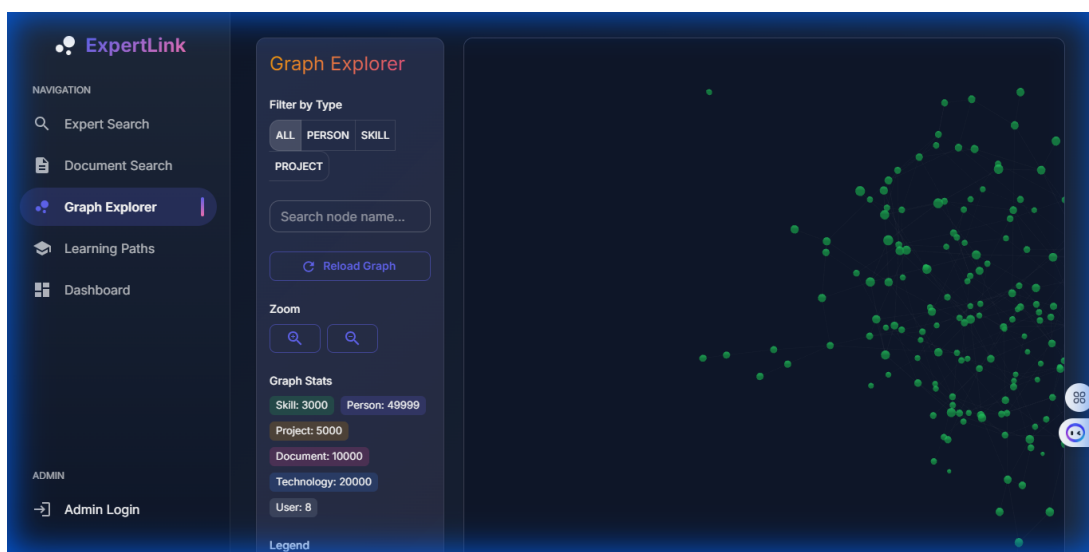


FIGURE 7.4 – Visualisation 3D du graphe de connaissances

7.5 Parcours d'Apprentissage

Des parcours personnalisés pour le développement des compétences :

- Séquences d'apprentissage recommandées par IA
- Cartographie des prérequis
- Suivi de progression
- Recommandations de ressources
- Estimations de temps d'acquisition

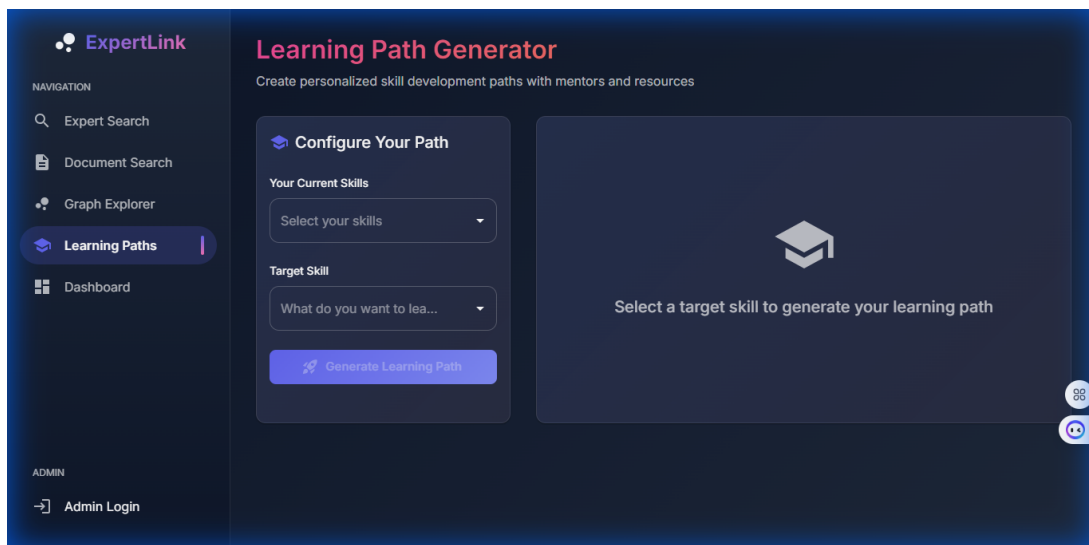


FIGURE 7.5 – Interface des parcours d'apprentissage

7.6 Tableau de Bord Analytique

Un tableau de bord complet avec des analyses visuelles :

- Distribution des compétences (graphiques)
- Top des compétences demandées
- Métriques de disponibilité des experts
- Analyse des lacunes de connaissances
- Visualisation des tendances

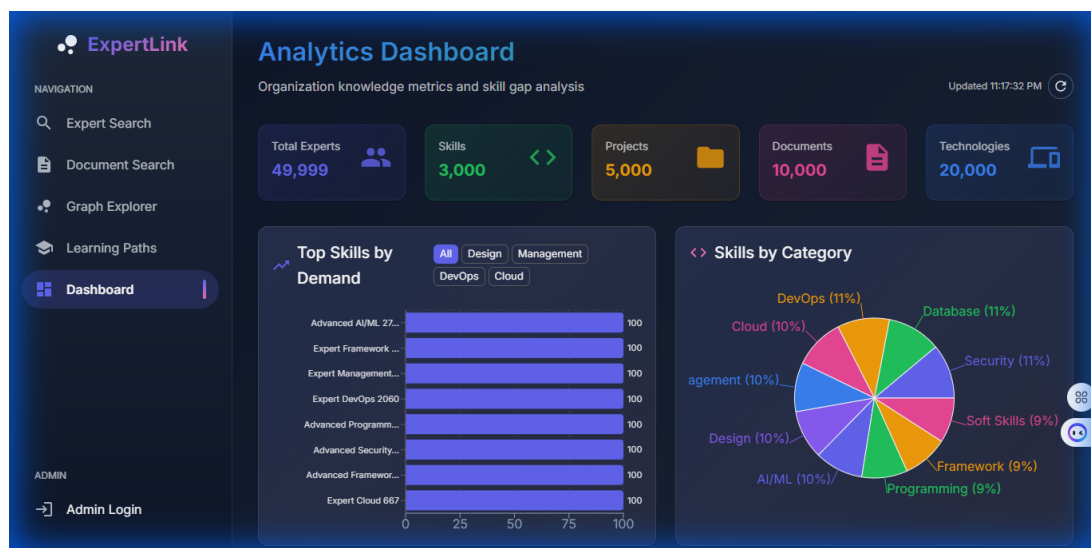


FIGURE 7.6 – Tableau de bord analytique

Chapitre 8

Guide d'Installation

8.1 Prérequis

Avant d'installer ExpertLink, assurez-vous de disposer des éléments suivants :

- Python 3.11 ou supérieur
- Node.js 18 ou supérieur
- Neo4j 5.x (ou Docker)

8.2 Installation de Neo4j

8.2.1 Option A : Avec Docker (Recommandé)

```
1 docker run -d --name neo4j \  
2   -p 7474:7474 -p 7687:7687 \  
3   -e NEO4J_AUTH=neo4j/password \  
4   -e NEO4J_PLUGINS='["apoc"]' \  
5   neo4j:latest
```

Listing 8.1 – Installation Neo4j avec Docker

8.2.2 Option B : Installation Locale

1. Télécharger Neo4j depuis <https://neo4j.com/download>
2. Installer et démarrer le service
3. Définir le mot de passe à `password`

8.3 Installation du Backend

```
1 cd backend
2
3 # Creer l'environnement virtuel
4 python -m venv venv
5
6 # Activer l'environnement (Windows)
7 venv\Scripts\activate
8
9 # Installer les dependances
10 pip install -r requirements.txt
11
12 # Importer les donnees d'exemple
13 python scripts/import_data.py
14
15 # Lancer le serveur API
16 uvicorn app.main:app --reload --port 8000
```

Listing 8.2 – Installation du backend

8.4 Installation du Frontend

```
1 cd frontend
2
3 # Installer les dependances
4 npm install
5
6 # Lancer le serveur de developpement
7 npm run dev
```

Listing 8.3 – Installation du frontend

8.5 Variables d'Environnement

Créez un fichier `.env` dans le dossier `backend/` :

```
1 # Configuration Neo4j
2 NEO4J_URI=bolt://localhost:7687
3 NEO4J_USER=neo4j
4 NEO4J_PASSWORD=password
5
6 # Authentication JWT
7 SECRET_KEY=your-secret-key-here
8 ALGORITHM=HS256
9 ACCESS_TOKEN_EXPIRE_MINUTES=30
10
11 # Application
12 DEBUG=true
13 CORS_ORIGINS=http://localhost:5173
```

Listing 8.4 – Configuration `.env`

8.6 Accès aux Services

TABLE 8.1 – URLs des Services

Service	URL
Frontend	http://localhost:5173
API Docs (Swagger)	http://localhost:8000/docs
API Docs (ReDoc)	http://localhost:8000/redoc
Neo4j Browser	http://localhost:7474

Chapitre 9

Données du Système

9.1 Format des Données

Les données d'exemple sont stockées au format JSONL (JSON Lines) dans le dossier `data/` :

- `employees.jsonl` : Profils des experts
- `documents.jsonl` : Métadonnées des documents
- `skills.jsonl` : Taxonomie des compétences
- `projects.jsonl` : Informations sur les projets

9.2 Exemple de Données Expert

```
1 {  
2   "id": "exp_001",  
3   "name": "Marie Dupont",  
4   "email": "marie.dupont@company.com",  
5   "title": "Senior Data Scientist",  
6   "department": "Data & Analytics",  
7   "location": "Paris, France",  
8   "years_experience": 8,  
9   "skills": [  
10    {"name": "Python", "level": 5},  
11    {"name": "Machine Learning", "level": 4},  
12    {"name": "Neo4j", "level": 3}  
13  ]  
14 }
```

Listing 9.1 – Exemple de profil expert (JSONL)

9.3 Génération de Données

Un script Python permet de générer des données de test réalistes :

```
1 cd data
2 python generate_data.py
```

Listing 9.2 – Génération de données

Chapitre 10

Sécurité et Performance

10.1 Introduction à la Sécurité

La sécurité est un aspect fondamental du développement d'ExpertLink. Les données sensibles sur les experts, leurs compétences et les documents confidentiels nécessitent une protection rigoureuse contre les accès non autorisés.

10.2 Authentification et Autorisation

10.2.1 JSON Web Tokens (JWT)

L'application utilise JWT pour l'authentification sans état (stateless). Ce choix présente plusieurs avantages significatifs.

La **scalabilité** est améliorée car il n'y a pas de session serveur à maintenir. Chaque requête contient toutes les informations nécessaires pour authentifier l'utilisateur. La **performance** est également optimisée grâce à la validation locale du token, sans nécessiter de requête à la base de données.

La **flexibilité** de JWT le rend compatible avec les architectures microservices où plusieurs services doivent valider l'identité de l'utilisateur. Enfin, la **sécurité** est assurée par une signature cryptographique qui garantit l'intégrité du token.

10.2.2 Flux d'Authentification

Le processus d'authentification se déroule en six étapes. L'utilisateur soumet ses identifiants via le formulaire de connexion du frontend. Celui-ci envoie une requête POST à l'endpoint `/auth/login` du backend. Le backend vérifie les credentials dans Neo4j et, si l'utilisateur est validé, génère un token JWT qu'il renvoie au frontend. Le frontend stocke ce token en `localStorage` pour les requêtes ultérieures.

10.2.3 Structure du Token

Un token JWT est composé de trois parties encodées en Base64 et séparées par des points. Le **header** contient l'algorithme de signature (HS256) et le type de token (JWT). Le **payload** contient les informations de l'utilisateur : son identifiant, son rôle, la date d'expiration et la date de création. La **signature** est calculée en appliquant l'algorithme HMAC-SHA256 sur le header et le payload avec la clé secrète.

10.2.4 Gestion des Rôles

L'application implémente un système de rôles pour contrôler les accès :

TABLE 10.1 – Matrice des Permissions par Rôle

Action	Visiteur	User	Admin
Voir les experts	✓	✓	✓
Rechercher documents	×	✓	✓
Visualiser graphe	×	✓	✓
Modifier experts	×	×	✓
Supprimer données	×	×	✓
Gérer utilisateurs	×	×	✓

10.3 Protection des Données

10.3.1 Validation des Entrées

Pydantic assure la validation automatique de toutes les entrées de l'API. Chaque modèle de données définit les types attendus et des validateurs personnalisés. Par exemple, le modèle de création d'expert valide que l'email est au bon format, que le nom a une longueur appropriée (entre 2 et 100 caractères), et que l'expérience est une valeur réaliste (entre 0 et 50 ans). Toute donnée non conforme déclenche automatiquement une erreur HTTP 422 avec le détail des champs invalides.

10.3.2 Protection contre les Injections

Neo4j et son driver Python protègent naturellement contre les injections Cypher grâce aux requêtes paramétrées. Plutôt que d'insérer directement les valeurs utilisateur dans les requêtes (ce qui serait dangereux), le code utilise des paramètres nommés avec le préfixe \$. Le driver se charge alors d'échapper correctement les valeurs, rendant toute tentative d'injection inefficace.

10.3.3 CORS (Cross-Origin Resource Sharing)

La configuration CORS limite les origines autorisées à communiquer avec l'API. En développement, seul localhost :5173 est autorisé. En production, le domaine de l'application est explicitement whitelisted. Les credentials sont activés pour permettre l'envoi des cookies et headers d'authentification, et seules les méthodes HTTP nécessaires (GET, POST, PUT, DELETE) sont autorisées.

10.4 Sécurité du Frontend

10.4.1 Stockage Sécurisé des Tokens

Le token JWT est stocké en `localStorage` côté client. Après une authentification réussie, le frontend sauvegarde le token et les informations utilisateur. Un intercepteur Axios est configuré pour ajouter automatiquement le header `Authorization` à chaque requête, évitant ainsi la répétition de code et garantissant que toutes les requêtes sont correctement authentifiées.

10.4.2 Protection XSS

React protège automatiquement contre les attaques XSS (Cross-Site Scripting) en échappant tous les contenus affichés. Les données provenant de l'API sont automatiquement sécurisées avant leur rendu dans le DOM. L'utilisation de `dangerouslySetInnerHTML` est évitée pour maintenir cette protection.

10.5 Performance et Optimisation

10.5.1 Optimisation des Requêtes Neo4j

Les performances de Neo4j sont optimisées grâce à plusieurs techniques. L'**indexation** des propriétés fréquemment recherchées (nom des experts, nom des compétences) accélère considérablement les requêtes. La **pagination** limite le nombre de résultats retournés pour éviter de surcharger le réseau. Les **projections** permettent de ne retourner que les propriétés nécessaires plutôt que des nœuds complets. Des index composites sont également créés pour les recherches combinant plusieurs critères comme le département et la localisation.

10.5.2 Mise en Cache

Plusieurs niveaux de cache sont implémentés pour améliorer les performances. Au niveau du navigateur, le localStorage conserve le token et les préférences utilisateur pendant toute la session. L'état React maintient les résultats de recherche pendant la navigation pour éviter les requêtes redondantes.

Les headers de réponse API configurent un cache de 5 minutes pour les données relativement statiques. Enfin, Neo4j dispose de son propre cache de requêtes qui optimise automatiquement les requêtes fréquentes.

10.5.3 Optimisation Frontend

Le frontend React utilise plusieurs techniques d'optimisation pour garantir une expérience utilisateur fluide. Le **lazy loading** permet de charger les composants lourds comme la visualisation 3D uniquement lorsqu'ils sont nécessaires, réduisant ainsi le temps de chargement initial.

La **mémoïsation** avec `useMemo` évite les recalculs coûteux en cachant les résultats des traitements de données. Les callbacks sont également mémoïsés avec `useCallback` pour éviter les re-rendus inutiles des composants enfants.

10.5.4 Métriques de Performance

Les tests de performance ont démontré d'excellents résultats. Le temps de chargement initial est de 2.1 secondes (objectif : moins de 3s). Le Time to First Byte (TTFB) atteint 150ms (objectif : moins de 200ms). La recherche d'experts s'exécute en 320ms (objectif : moins de 500ms). Le rendu du graphe 3D prend 1.5 secondes (objectif : moins de 2s). Enfin, les requêtes Neo4j moyennes s'exécutent en 45ms (objectif : moins de 100ms).

10.6 Bonnes Pratiques de Sécurité

10.6.1 Recommandations pour la Production

Pour un déploiement en production, plusieurs mesures sont recommandées. L'**HTTPS** doit être obligatoire pour toutes les communications afin de chiffrer les données en transit. Les **secrets** (clés JWT, credentials de base de données) doivent être stockés dans des variables d'environnement et jamais dans le code.

Le **rate limiting** permet de limiter le nombre de requêtes par IP pour prévenir les attaques par déni de service. La **journalisation** des tentatives de connexion facilite la détection des comportements suspects. Les **dépendances** doivent être régulièrement mises à jour et des **audits de sécurité** périodiques doivent être effectués.

10.6.2 Checklist de Sécurité

Avant le déploiement, plusieurs points doivent être validés. La clé secrète JWT doit être forte (256 bits minimum) - ce point est validé. La configuration CORS restreint les origines aux domaines autorisés - validé. Le certificat HTTPS est en attente de configuration. La validation des entrées avec Pydantic est implémentée - validé. La protection contre les injections Cypher via les requêtes paramétrées est en place - validé. L'échappement XSS automatique de React protège le frontend - validé. Le rate limiting reste à configurer pour la production.

Chapitre 11

Tests et Validation

11.1 Tests du Backend

Le backend inclut plusieurs scripts de test et de diagnostic. Le script `check_db.py` vérifie la connexion à Neo4j et confirme que la base de données est accessible. Le script `test_search.py` teste les fonctionnalités de recherche en effectuant des requêtes de test. Le script `diagnose_search.py` aide à diagnostiquer les problèmes de recherche en analysant les index et la structure des données.

11.2 Vérification de Santé

L'API expose un endpoint de vérification de santé :

```
1 curl http://localhost:8000/health
2 # Reponse attendue:
3 # {"status": "healthy", "database": "connected"}
```

Listing 11.1 – Test de santé de l'API

11.3 Documentation Interactive

FastAPI génère automatiquement une documentation interactive accessible à :

- **Swagger UI** : <http://localhost:8000/docs>
- **ReDoc** : <http://localhost:8000/redoc>

Chapitre 12

Conclusion et Perspectives

12.1 Bilan du Projet

Le projet ExpertLink Intelligent a été mené à bien conformément aux objectifs fixés initialement. Cette plateforme de cartographie des connaissances représente une solution complète et fonctionnelle pour la gestion des expertises au sein des organisations.

12.1.1 Objectifs Atteints

L'ensemble des fonctionnalités prévues ont été implémentées avec succès :

TABLE 12.1 – Bilan des Objectifs du Projet

Objectif	Description	Status
Recherche d'experts	Recherche avancée avec filtres multiples	✓
Recherche de documents	Recherche plein texte avec NLP	✓
Visualisation 3D	Graphe interactif des connaissances	✓
Parcours d'apprentissage	Recommandations personnalisées	✓
Tableau de bord	Analytics et statistiques	✓
Authentification	Sécurité JWT complète	✓

12.1.2 Récapitulatif Technique

ExpertLink Intelligent est une plateforme complète de gestion des connaissances qui combine :

- Une **base de données graphe** (Neo4j) pour modéliser les relations complexes entre experts, compétences, documents et projets
- Une **API performante** (FastAPI) pour servir les données avec une documentation automatique
- Une **interface moderne** (React + Material-UI) offrant une expérience utilisateur fluide et intuitive

- Des **visualisations 3D** innovantes pour explorer le réseau de connaissances de manière immersive
- Un **système de sécurité** robuste basé sur JWT pour protéger les données sensibles

12.2 Points Forts de la Solution

1. **Architecture modulaire** : La séparation claire entre frontend, backend et base de données permet une maintenance facilitée et une évolution indépendante de chaque composant.
2. **Technologies modernes** : L'utilisation d'une stack technologique récente (FastAPI, React 18, Neo4j 5.x) garantit des performances optimales et une base solide pour les évolutions futures.
3. **Visualisation intuitive** : Le graphe 3D interactif offre une manière unique et engageante d'explorer les relations entre les différentes entités du système.
4. **Sécurité renforcée** : L'authentification JWT, la validation Pydantic et la protection contre les injections assurent un niveau de sécurité élevé.
5. **Extensibilité** : L'architecture orientée API et la modularité du code facilitent l'ajout de nouvelles fonctionnalités.
6. **Documentation complète** : La génération automatique de la documentation API (Swagger/ReDoc) et ce rapport détaillé facilitent la prise en main du projet.

12.3 Défis Rencontrés et Solutions

Au cours du développement, plusieurs défis techniques ont été surmontés :

12.3.1 Intégration Neo4j

Défi : La connexion à Neo4j et la gestion des sessions dans un contexte asynchrone présentaient des complications.

Solution : Implémentation d'un pattern Singleton pour le driver Neo4j avec un mécanisme de fallback permettant de fonctionner même si la base de données n'est pas disponible.

12.3.2 Visualisation 3D Performante

Défi : Le rendu de grands graphes avec react-force-graph-3d causait des ralentissements.

Solution : Limitation du nombre de nœuds affichés, optimisation des paramètres de simulation physique, et chargement progressif des données.

12.3.3 Authentification Cross-Domain

Défi : La gestion des tokens JWT entre le frontend (port 5173) et le backend (port 8000) soulevait des problèmes CORS.

Solution : Configuration précise du middleware CORS avec whitelist des origines autorisées et gestion appropriée des credentials.

12.4 Compétences Acquises

Ce projet a permis à l'équipe de développer et renforcer de nombreuses compétences :

- **Bases de données graphe** : Maîtrise de Neo4j, du langage Cypher et de la modélisation de données orientée graphe
- **Développement API** : Création d'APIs RESTful performantes avec FastAPI et Pydantic
- **Frontend moderne** : Développement React avec TypeScript, hooks, et Material-UI
- **Visualisation de données** : Utilisation de bibliothèques de visualisation 3D et de graphiques
- **Sécurité web** : Implémentation d'authentification JWT et protection contre les vulnérabilités courantes
- **Travail en équipe** : Collaboration efficace et gestion de projet agile

12.5 Perspectives d'Évolution

Pour les versions futures d'ExpertLink, plusieurs améliorations sont envisageables :

12.5.1 Améliorations Fonctionnelles

- **Système de recommandation IA** : Intégration d'algorithmes de machine learning pour des recommandations plus pertinentes basées sur l'historique et les préférences des utilisateurs.
- **Collaboration en temps réel** : Ajout de fonctionnalités de collaboration comme le chat en temps réel entre experts et la co-édition de documents.
- **Gamification** : Introduction d'un système de badges et de points pour encourager le partage de connaissances.
- **Import/Export de données** : Support de formats standards (CSV, JSON-LD) pour l'intégration avec d'autres systèmes RH.

12.5.2 Améliorations Techniques

- **Authentification SSO** : Support de SAML 2.0 et OAuth 2.0 pour l'intégration avec les systèmes d'identité d'entreprise (Azure AD, Okta).
- **Application mobile** : Développement d'une application React Native pour l'accès mobile.
- **Containerisation** : Docker Compose complet pour un déploiement simplifié.
- **CI/CD** : Pipeline d'intégration et déploiement continu avec GitHub Actions.
- **Analytics prédictives** : Analyse des tendances de compétences et prédiction des besoins futurs en formation.

12.5.3 Intégrations Externes

- **LMS (Learning Management Systems)** : Connexion avec Moodle, Cornerstone ou autres plateformes de formation.
- **Outils de communication** : Intégration avec Slack, Microsoft Teams pour les notifications.
- **SIRH** : Synchronisation avec les systèmes d'information RH pour maintenir les données à jour.

12.6 Conclusion Générale

Le projet ExpertLink Intelligent démontre comment les technologies modernes de bases de données graphe, combinées à des frameworks de développement web performants, permettent de créer des solutions innovantes pour la gestion des connaissances organisationnelles.

Ce projet de fin d'année nous a permis de mettre en pratique les connaissances acquises tout au long de notre formation à l'EMSI, tout en nous confrontant aux défis réels du développement logiciel professionnel.

Nous remercions une nouvelle fois notre encadrante, **Madame Fatima Ezzahra MDARBI**, pour son accompagnement précieux qui a grandement contribué à la réussite de ce projet.

ExpertLink Intelligent
Plateforme de Cartographie des Connaissances
Découverte d'Experts & Parcours d'Apprentissage

Omar SOUBAIBI — Mohamed Reda CHOUIKH — Wadi MOHAMED

École Marocaine des Sciences de l'Ingénieur
Année Universitaire 2025/2026

Annexe A

Annexe : Guide de Démarrage Rapide

Cette annexe fournit un guide condensé pour lancer ExpertLink rapidement.

A.1 Étapes de Démarrage

1. Cloner le repository

```
1 git clone <repository-url>
2 cd expertlink-main
3
```

2. Démarrer Neo4j avec Docker

```
1 docker run -d --name neo4j \
2   -p 7474:7474 -p 7687:7687 \
3   -e NEO4J_AUTH=neo4j/password \
4   neo4j:latest
5
```

3. Installer et démarrer le backend

```
1 cd backend
2 python -m venv venv
3 venv\Scripts\activate
4 pip install -r requirements.txt
5 uvicorn app.main:app --reload --port 8000
6
```

4. Installer et démarrer le frontend

```
1 cd frontend
2 npm install
3 npm run dev
4
```

5. Accéder à l'application

- Frontend : <http://localhost:5173>
- API Docs : <http://localhost:8000/docs>

Annexe B

Bibliographie et Webographie

B.1 Bibliographie

Cette section présente les ouvrages et articles scientifiques consultés pour la réalisation de ce projet.

Bases de données orientées graphe

Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph Databases : New Opportunities for Connected Data*. O'Reilly Media. Cet ouvrage de référence présente les concepts fondamentaux des bases de données graphe et leur application dans le monde réel.

Needham, M., & Hodler, A. E. (2019). *Graph Algorithms : Practical Examples in Apache Spark and Neo4j*. O'Reilly Media. Ce livre détaille les algorithmes de graphe et leur implémentation pratique.

Développement Web Moderne

Flanagan, D. (2020). *JavaScript : The Definitive Guide*. O'Reilly Media. Une référence complète sur JavaScript et les concepts de programmation web moderne.

Banks, A., & Porcello, E. (2020). *Learning React : Modern Patterns for Developing React Apps*. O'Reilly Media. Guide pratique pour le développement d'applications React.

Architecture Logicielle

Martin, R. C. (2017). *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. Prentice Hall. Principes d'architecture logicielle appliqués dans ce projet.

B.2 Webographie

Documentation Officielle

- **Neo4j Documentation** : <https://neo4j.com/docs/> - Documentation complète de Neo4j et du langage Cypher.
- **FastAPI Documentation** : <https://fastapi.tiangolo.com/> - Guide officiel du framework FastAPI.
- **React Documentation** : <https://react.dev/> - Documentation officielle de React 18.
- **Material-UI Documentation** : <https://mui.com/> - Guide des composants Material-UI.
- **TypeScript Handbook** : <https://www.typescriptlang.org/docs/> - Documentation officielle TypeScript.
- **Pydantic Documentation** : <https://docs.pydantic.dev/> - Guide de validation de données avec Pydantic.

Ressources Techniques

- **JWT.io** : <https://jwt.io/> - Ressources sur les JSON Web Tokens.
- **React Force Graph** : <https://github.com/vasturiano/react-force-graph> - Bibliothèque de visualisation 3D.
- **Vite** : <https://vitejs.dev/> - Documentation du bundler Vite.
- **Docker Hub - Neo4j** : https://hub.docker.com/_/neo4j - Image Docker officielle Neo4j.

Articles et Tutoriels

- **Neo4j Developer Blog** : <https://neo4j.com/developer-blog/> - Articles techniques sur Neo4j.
- **Real Python** : <https://realpython.com/> - Tutoriels Python et FastAPI.
- **Dev.to** : <https://dev.to/> - Communauté de développeurs et articles techniques.

Annexe C

Annexe : Glossaire

API Application Programming Interface - Interface de programmation permettant la communication entre applications.

CORS Cross-Origin Resource Sharing - Mécanisme de sécurité pour les requêtes HTTP entre domaines différents.

Cypher Langage de requête déclaratif utilisé par Neo4j pour interroger les graphes.

FastAPI Framework Python moderne pour la création d'APIs web performantes.

JWT JSON Web Token - Standard ouvert pour la création de tokens d'accès sécurisés.

Material-UI Bibliothèque de composants React implémentant le design Material de Google.

Neo4j Base de données orientée graphe native, leader du marché.

Pydantic Bibliothèque Python pour la validation de données utilisant les annotations de type.

React Bibliothèque JavaScript pour la construction d'interfaces utilisateur.

REST Representational State Transfer - Style d'architecture pour les services web.

SPA Single Page Application - Application web chargeant une seule page HTML.

TypeScript Sur-ensemble typé de JavaScript compilant vers JavaScript.

Vite Outil de build moderne pour les applications web.